LAB 3

Component Modelling & Architectural Pattern Selection

Problem Statement 4 — Student Project Portfolio & Showcase App

Field	Details
Name	Porla Neha
SRN	PES1UG25AM808
Section	C
Semester	5
Project	Student Project Portfolio & Showcase App
Selected Functional Requirement	FR-005 — Project Leaderboard Management (Epic SCRUM-7, Lab 2)
Related Non-Functional Requirements	NFR-001 (Performance & Security), NFR-002 (Availability & Reliability)
Selected Architecture	Microservices Architecture

1. Objective

To evaluate the Layered, Microservices and Client-Server architectural styles, select the most suitable one for the Project Leaderboard Management functionality of the Student Project Portfolio & Showcase App, and model it as a UML Component Diagram showing components, provided/required interfaces and dependencies. The functional requirement (FR-005) carried forward from Labs 1 and 2 is used as the basis for this architecture.

2. Scenario Review

Lab 1 defined the requirements of the showcase app (use case: Submit Multimedia Artifacts), and Lab 2 converted FR-005 into the Project Leaderboard Management Epic (SCRUM-7) with four user stories (SCRUM-8 to SCRUM-11, 21 story points). Lab 3 designs the software architecture that realises these stories.

Key functional requirements (FR-005 / SCRUM-8 to 11)

- Calculate project rankings from judge evaluation scores and valid public votes (SCRUM-9).
- Display the leaderboard to public users (SCRUM-8).
- Reflect newly submitted valid scores and votes in the leaderboard (SCRUM-10).
- Exclude invalid or duplicate votes from all calculations (SCRUM-11, anti-sybil rules from FR-004).

Key non-functional requirements

- NFR-001: API response latency under 300 ms under simulated peak showcase load, with authentication and anti-sybil controls enforced.
- NFR-002: At least 99.5% availability during the showcase, and no loss of committed scores or votes after recoverable failures.

Main challenges

- **Performance:** leaderboard reads and vote updates spike when many users view the showcase at once.
- **Security:** only verified users may vote, and duplicate or fake votes must never reach the ranking calculation.
- **Usability / consistency:** the displayed ranking must always match the latest valid scores and votes.

3. Architectural Style Analysis

Style	Pros	Cons
Layered	Simple; clear separation of layers; easy consistency.	Scaled as one unit; a voting spike cannot be scaled alone; tight coupling.
Microservices	Services scale independently; failures are isolated.	Network latency; operational complexity.
Client-Server	Central control; simple deployment.	Single point of failure; scalability bottleneck.

4. Architecture Selection & Component Identification

Selected architecture: Microservices Architecture. Leaderboard Service is the central coordinator (like the Order Manager) and Vote Validation Service is the supporting service (like the Payment Service).

Component	Responsibility
Leaderboard UI	Shows the leaderboard to public users (SCRUM-8).
Leaderboard Service	Handles leaderboard queries and requests rankings (SCRUM-8, 10).
Vote Validation Service	Accepts valid votes, rejects invalid or duplicate ones (SCRUM-11).
Ranking Calculation Service	Calculates rankings from judge scores and valid votes (SCRUM-9).
Leaderboard Database	Stores valid votes, judge scores and ranking inputs.

5. UML Component Diagram

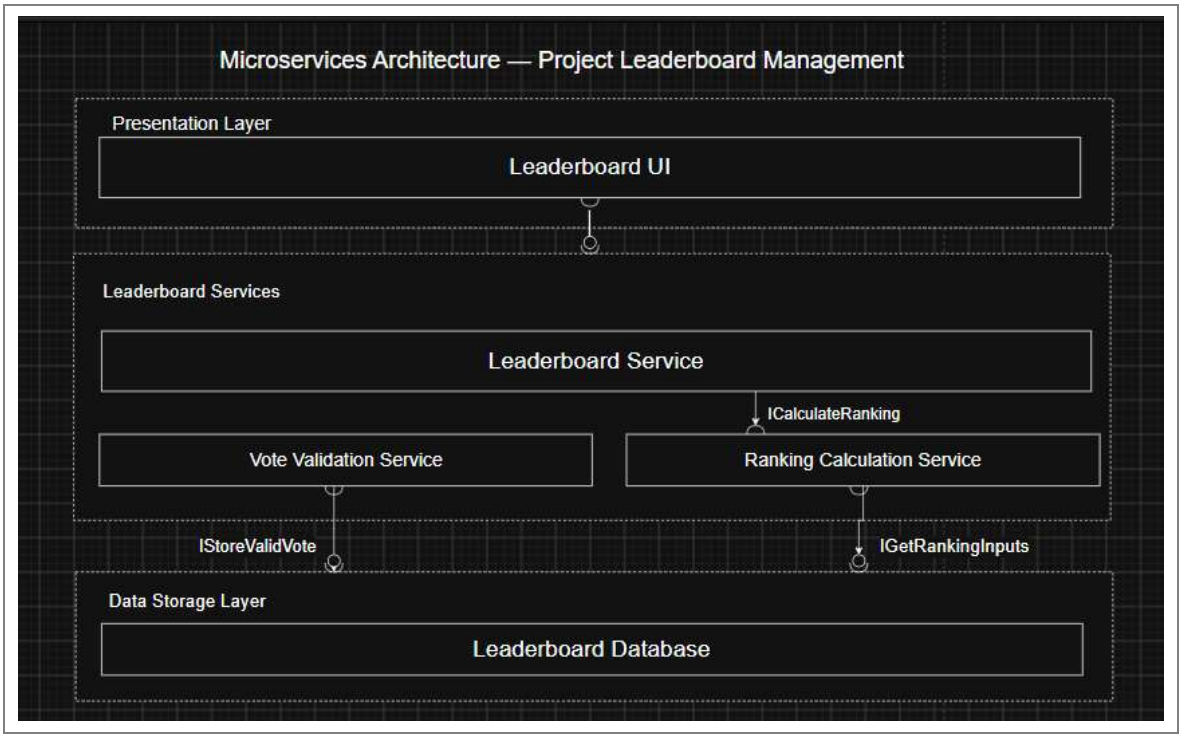


Figure 1. UML Component Diagram — Project Leaderboard Management (Microservices Architecture).

Interfaces Shown

Interface	Provided by	Required by	Technology
ILeaderboardQuery	Leaderboard Service	Leaderboard UI	REST / HTTPS
ICalculateRanking	Ranking Calculation Service	Leaderboard Service	REST / JSON
IStoreValidVote	Leaderboard Database	Vote Validation Service	SQL write
IGetRankingInputs	Leaderboard Database	Ranking Calculation Service	SQL read

6. Written Justification

Architecture Selection: We chose Microservices Architecture for the Student Project Portfolio & Showcase App (Project Leaderboard Management) System.

Architectural Choice: Microservices Architecture, with vote validation and ranking calculation as separate, independently deployable services.

Two Reasons:

1. **Independent scaling.** Voting and viewing spike during the showcase, so Vote Validation and Ranking Calculation can be scaled alone to meet the 300 ms target (NFR-001).
2. **Fault isolation.** If the Ranking Calculation Service fails, valid votes are still stored and rankings are recalculated on recovery, supporting 99.5% availability (NFR-002).

Security Advantage: Only votes that pass the anti-sybil checks in the Vote Validation Service are stored, so fake or duplicate votes never reach the ranking.

Performance Benefit: Ranking calculation runs in its own service, so it does not slow down vote validation or leaderboard queries.

7. Conclusion

Microservices Architecture was selected for Project Leaderboard Management (FR-005). The component diagram has five components and four interfaces, and links back to the Lab 1 requirements and the Lab 2 Epic SCRUM-7.